

# Shot Quality / Possession Value Demo

Caleb Ramsey

2026-05-23

## What This Is

This is a quick proof-of-concept for automating a shot quality / possession value workflow from public NBA data.

The goal right now is simple:

1. pull real NBA games
2. pull real shot events
3. assign a basic public-data shot quality bucket
4. summarize it game-to-game
5. show what can be automated now vs what probably still needs film/manual tracking

This is **not** the finished model. It is the working shell.

## Basic Code Flow

```
NBA game logs
  -> recent player games
NBA play-by-play
  -> real field-goal attempts
Proxy rules
  -> 9 / 7 / 5 / 3 / Prayer buckets
Summary tables
  -> bucket distribution + game-by-game SQ
Possession layer
  -> AST, TO, STL, BLK, OREB, fouls
```

## Helper Code

```
safe_num <- function(x) suppressWarnings(as.numeric(x))

safe_pct <- function(num, den) {
  ifelse(is.na(den) | den == 0, NA_real_, num / den)
}

extract_table <- function(raw, table_name) {
```

```

if (is.list(raw) && table_name %in% names(raw)) {
  raw[[table_name]] %>% janitor::clean_names()
} else if (is.data.frame(raw)) {
  raw %>% janitor::clean_names()
} else {
  tibble()
}
}

parse_clock_seconds <- function(clock) {
  mins <- stringr::str_match(clock, "PT(\\d+)M")[, 2] %>% safe_num()
  secs <- stringr::str_match(clock, "M([0-9.]+)S")[, 2] %>% safe_num()
  mins <- tidyr::replace_na(mins, 0)
  secs <- tidyr::replace_na(secs, 0)
  mins * 60 + secs
}

```

## Shot Quality Rules

These are basic placeholder rules so we can demo the structure. They can be replaced with the real film-based rules later.

```

assign_sq_bucket <- function(shot_value, shot_distance, action_type, sub_type, clock_seconds) {
  action_text <- stringr::str_to_lower(stringr::str_c(action_type, sub_type, sep = " "))

  dplyr::case_when(
    clock_seconds <= 2 ~ "Prayer",

    stringr::str_detect(action_text, "dunk|layup|alley oop") &
      shot_distance <= 3 ~ "9",

    shot_value == 3 &
      shot_distance >= 24 &
      stringr::str_detect(action_text, "catch|jump") ~ "7",

    shot_value == 2 & shot_distance <= 8 ~ "7",
    shot_value == 3 ~ "5",
    shot_distance > 8 & shot_distance <= 18 ~ "5",
    shot_distance > 18 & shot_value == 2 ~ "3",
    TRUE ~ "3"
  )
}

sq_bucket_value <- function(bucket) {
  dplyr::case_when(
    bucket == "9" ~ 9,
    bucket == "7" ~ 7,
    bucket == "5" ~ 5,
    bucket == "3" ~ 3,
    bucket == "1" ~ 1,
    bucket == "Prayer" ~ 9,
    TRUE ~ NA_real_
  )
}

```

```

}

score_player_shots <- function(player_shots) {
  player_shots %>%
    dplyr::mutate(
      sq_bucket_raw = assign_sq_bucket(
        shot_value = shot_value,
        shot_distance = shot_distance,
        action_type = action_type,
        sub_type = sub_type,
        clock_seconds = clock_seconds
      ),
      late_clock = clock_seconds <= 7 & clock_seconds > 2,
      prayer_context = clock_seconds <= 2,
      sq_bucket = dplyr::case_when(
        prayer_context ~ "Prayer",
        late_clock & sq_bucket_raw == "3" ~ "5",
        late_clock & sq_bucket_raw == "5" ~ "7",
        TRUE ~ sq_bucket_raw
      ),
      sq_value = sq_bucket_value(sq_bucket),
      made_points = dplyr::if_else(shot_made, shot_value, 0),
      sq_result_value = dplyr::case_when(
        sq_bucket == "Prayer" ~ made_points,
        shot_made ~ made_points,
        !shot_made & sq_bucket == "9" ~ -0.5,
        !shot_made & sq_bucket == "7" ~ -1,
        !shot_made & sq_bucket == "5" ~ -2,
        !shot_made & sq_bucket == "3" ~ -4,
        !shot_made & sq_bucket == "1" ~ -6,
        TRUE ~ 0
      )
    )
}

```

## Data Pull Functions

```

get_player_recent_games <- function(player_id, season, season_type, n_games = 5) {
  raw <- nba_playergamelog(
    player_id = player_id,
    season = season,
    season_type = season_type
  )

  out <- extract_table(raw, "PlayerGameLog")
  if (nrow(out) == 0) return(tibble())

  out %>%
    dplyr::mutate(
      player_id = as.character(player_id),
      game_id = as.character(game_id),
      game_date = lubridate::mdy(game_date),

```

```

    dplyr::across(
      dplyr::any_of(c(
        "pts", "reb", "ast", "stl", "blk", "tov", "pf",
        "fga", "fgm", "fta", "ftm", "oreb", "dreb", "fg3a", "fg3m",
        "plus_minus"
      )),
      safe_num
    )
  ) %>%
  dplyr::arrange(dplyr::desc(game_date)) %>%
  dplyr::slice_head(n = n_games)
}

get_player_game_logs_all <- function(player_id, season, season_type) {
  raw <- nba_playergamelog(
    player_id = player_id,
    season = season,
    season_type = season_type
  )

  out <- extract_table(raw, "PlayerGameLog")
  if (nrow(out) == 0) return(tibble())

  out %>%
    dplyr::mutate(
      player_id = as.character(player_id),
      game_id = as.character(game_id),
      game_date = lubridate::mdy(game_date),
      dplyr::across(
        dplyr::any_of(c(
          "pts", "reb", "ast", "stl", "blk", "tov", "pf",
          "fga", "fgm", "fta", "ftm", "oreb", "dreb", "fg3a", "fg3m",
          "plus_minus"
        )),
        safe_num
      )
    )
}

get_player_pbp_shots <- function(game_id, target_player_id) {
  pbp_raw <- tryCatch({
    nba_playbyplayv3(game_id = game_id) %>%
      janitor::clean_names()
  }, error = function(e) NULL)

  if (is.null(pbp_raw) || !"play_by_play" %in% names(pbp_raw)) return(tibble())

  pbp_tbl <- pbp_raw$play_by_play
  if (is.null(pbp_tbl) || !is.data.frame(pbp_tbl) || nrow(pbp_tbl) == 0) return(tibble())

  pbp_tbl %>%
    janitor::clean_names() %>%
    dplyr::filter(

```

```

    is_field_goal == 1,
    !is.na(person_id),
    as.character(person_id) == as.character(target_player_id)
  ) %>%
  dplyr::transmute(
    game_id = as.character(game_id),
    event_num = action_number,
    period,
    clock,
    clock_seconds = parse_clock_seconds(clock),
    player_id = as.character(person_id),
    player_name = stringr::str_squish(player_name),
    team = team_tricode,
    shot_result,
    shot_made = shot_result == "Made",
    shot_value = safe_num(shot_value),
    shot_distance = safe_num(shot_distance),
    action_type = stringr::str_squish(action_type),
    sub_type = stringr::str_squish(sub_type),
    description = stringr::str_squish(description),
    x = safe_num(x_legacy),
    y = safe_num(y_legacy),
    points_total = safe_num(points_total)
  )
}

get_player_shot_events <- function(player_id, game_ids) {
  game_ids %>%
  purrr::map_dfr(~ get_player_pbp_shots(.x, player_id))
}

```

## One Player Example

This is the basic working example for Victor Wembanyama.

### Shot Quality Buckets

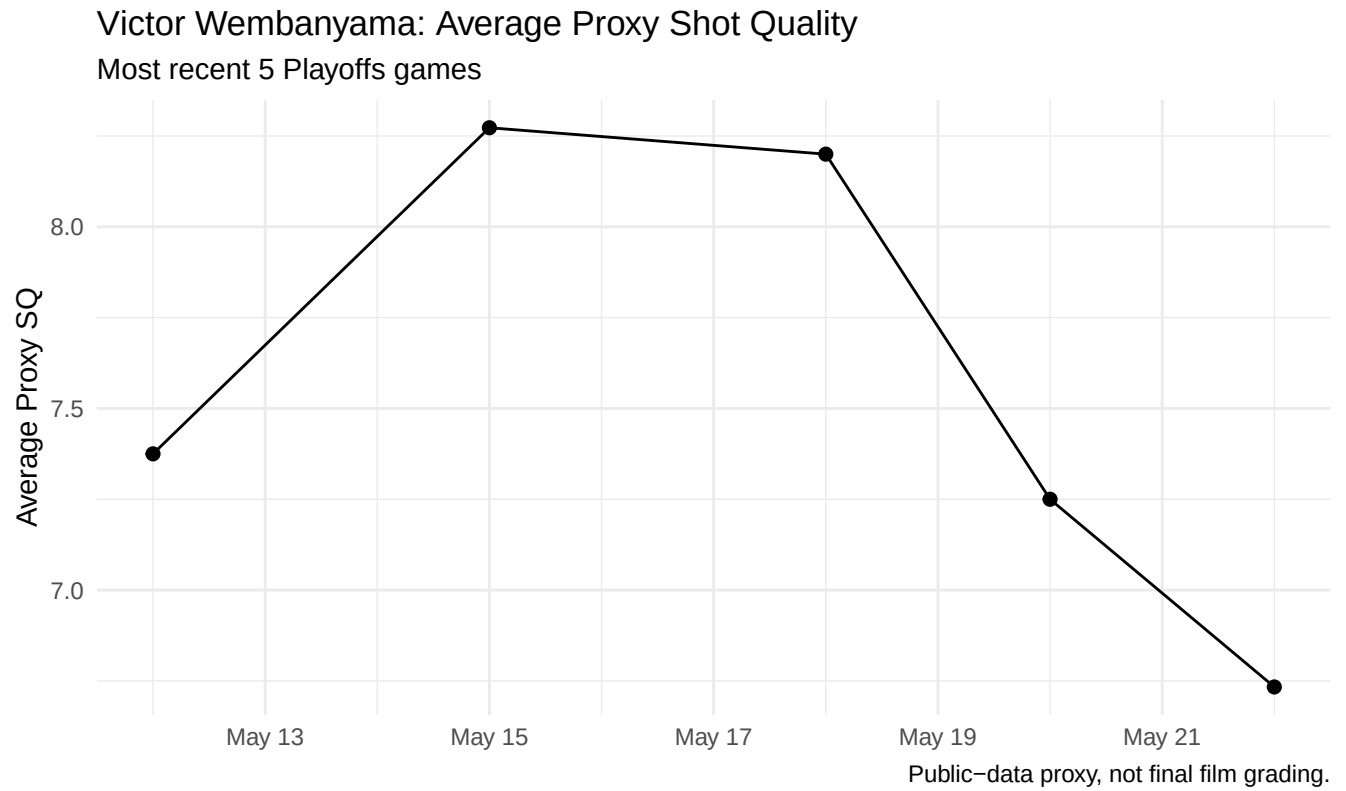
Table 1: Victor Wembanyama: Proxy shot quality bucket distribution

| sq_bucket | SQ_Value | FGA | FGM | FG_PCT | Points | Avg_Shot_Distance | SQ_Result_Value | Frequency |
|-----------|----------|-----|-----|--------|--------|-------------------|-----------------|-----------|
| 9         | 9        | 36  | 27  | 75.0%  | 54     | 1.3               | 49.5            | 43.4%     |
| Prayer    | 9        | 1   | 0   | 0.0%   | 0      | 4.0               | 0.0             | 1.2%      |
| 7         | 7        | 35  | 15  | 42.9%  | 37     | 16.7              | 17.0            | 42.2%     |
| 5         | 5        | 10  | 3   | 30.0%  | 7      | 9.2               | -7.0            | 12.0%     |
| 3         | 3        | 1   | 0   | 0.0%   | 0      | 19.0              | -4.0            | 1.2%      |

### Game-by-Game View

Table 2: Victor Wembanyama: Game-by-game proxy SQ

| Date       | Matchup     | Result | PTS | REB | AST | TOV | FGA | Avg SQ | SQ Value | Late Clock | Prayer |
|------------|-------------|--------|-----|-----|-----|-----|-----|--------|----------|------------|--------|
| 2026-05-22 | SAS vs. OKC | L      | 26  | 4   | 3   | 2   | 15  | 6.73   | 7.0      | 1          | 0      |
| 2026-05-20 | SAS @ OKC   | L      | 21  | 17  | 6   | 4   | 16  | 7.25   | 9.5      | 0          | 0      |
| 2026-05-18 | SAS @ OKC   | W      | 41  | 24  | 3   | 3   | 25  | 8.20   | 19.5     | 0          | 1      |
| 2026-05-15 | SAS @ MIN   | W      | 19  | 6   | 2   | 4   | 11  | 8.27   | 8.0      | 0          | 0      |
| 2026-05-12 | SAS vs. MIN | W      | 27  | 17  | 5   | 4   | 16  | 7.38   | 11.5     | 0          | 0      |



## Possession Value Layer

This is a rough add-on layer using stats that are already easy to pull.

Table 3: Victor Wembanyama: SQ + possession value demo

| Date       | Matchup     | Result | PTS | AST | TOV | STL | BLK | OREB | Box PV | SQ Value | Total PV | Avg SQ |
|------------|-------------|--------|-----|-----|-----|-----|-----|------|--------|----------|----------|--------|
| 2026-05-22 | SAS vs. OKC | L      | 26  | 3   | 2   | 1   | 2   | 0    | 1.0    | 7.0      | 8.0      | 6.73   |
| 2026-05-20 | SAS @ OKC   | L      | 21  | 6   | 4   | 1   | 4   | 5    | 9.2    | 9.5      | 18.8     | 7.25   |
| 2026-05-18 | SAS @ OKC   | W      | 41  | 3   | 3   | 1   | 3   | 9    | 10.8   | 19.5     | 30.2     | 8.20   |
| 2026-05-15 | SAS @ MIN   | W      | 19  | 2   | 4   | 0   | 3   | 1    | -3.2   | 8.0      | 4.8      | 8.27   |
| 2026-05-12 | SAS vs. MIN | W      | 27  | 5   | 4   | 0   | 3   | 2    | 3.0    | 11.5     | 14.5     | 7.38   |

## Quick Scale Example

This optional section runs the same SQ idea on small playoff groups.

For a quick meeting version, use `comparison_top_n: 3` and `n_recent_games: 3`.

`## Comparison section skipped for speed. Set run_comparison: true in params when ready.`

## What We Can Calculate Now

Table 4: Automation map

| Metric                    | Status                      | Note                                                             |
|---------------------------|-----------------------------|------------------------------------------------------------------|
| Shot distance             | Automatable                 | Available in play-by-play / shot data.                           |
| Shot value                | Automatable                 | 2 vs 3 is clean.                                                 |
| Shot type/subtype         | Automatable-ish             | Useful, but labels are not perfect.                              |
| Late clock                | Proxy-able                  | Clock is available; true possession context may need refinement. |
| Prayer shots              | Proxy-able                  | Can use clock + distance rules.                                  |
| Free throws               | Automatable                 | Box score is clean; foul context is harder.                      |
| Assists                   | Automatable                 | Box score / tracking.                                            |
| Potential assists         | Trackable                   | Available through passing tracking if endpoint works.            |
| Turnovers                 | Automatable                 | Clean box-score event.                                           |
| Deflections               | Trackable                   | NBA hustle stats.                                                |
| Steals                    | Automatable                 | Clean possession gain.                                           |
| Blocks                    | Automatable                 | Need context for possession recovery.                            |
| Charges drawn             | Semi-trackable              | Hustle data may help; verify endpoint.                           |
| Allowed rebound           | Proxy-able                  | DRB% / lineup context, not exact assignment.                     |
| Blow-bys                  | Manual                      | Needs film charting.                                             |
| True contest quality      | Manual / tracking-dependent | Not reliably captured in basic public PBP.                       |
| Uncontested shots allowed | Manual / tracking-dependent | Needs film or reliable tracking source.                          |

## Takeaways

- We can already pull real games and real shot events.

- We can create a usable proxy SQ from public data.
- We can track player game-to-game movement.
- We can add a rough possession value layer now.
- The full model needs better contest data, league-average calibration, and some film/manual inputs.

## **Export**